

Newton-Raphson Method: Nonlinear Regression (Michaelis-Menten Model)

Sulagna Roy

September 13, 2026

Contents

1	Introduction	1
2	Step 1: Setup and Simulating Data	2
3	Step 2: The Objective Function Being Minimized	2
4	Step 3: Gradient and Hessian, Derived	2
5	Step 4: Implementing Newton-Raphson in R	3
6	Step 5: Numerical Results from This Run	5
7	Step 6: Visualizing the Fit	5
8	Conclusion	6

1 Introduction

This document explains a **two-parameter nonlinear regression** fit using the **Newton-Raphson method**. The model being fit is:

$$y_i = \frac{\theta_1 x_i}{\theta_2 + x_i} + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2)$$

This is the **Michaelis-Menten** equation, widely used in biochemistry (enzyme kinetics), pharmacology (dose-response), and growth modeling. It describes a relationship that rises quickly at first and then **saturates**, approaching an asymptote of θ_1 as $x \rightarrow \infty$, with θ_2 controlling how quickly that saturation happens (it is the x -value at which y reaches half of θ_1).

Unlike ordinary linear regression, this model is **nonlinear in its parameters** (θ_1, θ_2), so there is no closed-form least-squares solution. Newton-Raphson is used to numerically minimize the sum of squared residuals by iterating toward the values of θ_1, θ_2 that best fit the simulated data.

2 Step 1: Setup and Simulating Data

```
rm(list = ls())
n = 50
set.seed(123)
e = rnorm(n, 0, 0.2)
x = 1:n
m = 1000
theta = matrix(0, ncol = 2, nrow = m)
epsilon = 10^(-6)
theta[1, ] = c(5, 2)
y = (theta[1, 1] * x / (theta[1, 2] + x)) + e
```

What this code does:

- `rm(list = ls())` clears the workspace.
- `n = 50` sets the sample size to 50 observations.
- `set.seed(123)` fixes the random seed for reproducibility.
- `e = rnorm(n, 0, 0.2)` generates 50 independent normal errors with mean 0 and standard deviation 0.2 — this is the noise added to the “true” curve.
- `x = 1:n` creates the predictor as the integers 1 through 50.
- `m = 1000` sets the maximum number of Newton-Raphson iterations allowed.
- `theta = matrix(0, ncol = 2, nrow = m)` pre-allocates a 1000×2 matrix. Each **row** will hold one iteration’s estimate of (θ_1, θ_2) ; pre-allocating like this (rather than growing the matrix inside the loop) is efficient in R.
- `epsilon = 10^(-6)` is the convergence tolerance.
- `theta[1, ] = c(5, 2)` sets the **true parameter values** used to generate the data: $\theta_1 = 5$, $\theta_2 = 2$. Note this row is doing double duty — it is both “the truth used to simulate **y**” and, a moment later, “the starting value for the Newton-Raphson search.” Because the search is started exactly at the truth, this example mainly demonstrates the mechanics of the update step rather than testing robustness to a poor starting guess (see the note in Step 3).
- `y = (theta[1,1] * x / (theta[1,2] + x)) + e` generates the observed response: the Michaelis-Menten curve evaluated at the true $(\theta_1, \theta_2) = (5, 2)$, plus the random noise **e**.

3 Step 2: The Objective Function Being Minimized

Newton-Raphson here is minimizing the **residual sum of squares (RSS)**:

$$S(\theta_1, \theta_2) = \sum_{i=1}^n \left(y_i - \frac{\theta_1 x_i}{\theta_2 + x_i} \right)^2$$

This is equivalent to maximum likelihood estimation under the assumption of normal errors. To minimize S , Newton-Raphson finds a **stationary point of the gradient** $\nabla S = 0$ using the same update logic as in a likelihood setting: build a local quadratic (via the Hessian) and jump to its minimum, repeatedly.

4 Step 3: Gradient and Hessian, Derived

Let $\hat{y}_i(\theta) = \frac{\theta_1 x_i}{\theta_2 + x_i}$ and $r_i = y_i - \hat{y}_i(\theta)$ (the residual). Differentiating $S(\theta) = \sum r_i^2$:

Gradient (first derivatives):

$$\frac{\partial S}{\partial \theta_1} = -2 \sum r_i \cdot \frac{x_i}{\theta_2 + x_i}, \quad \frac{\partial S}{\partial \theta_2} = 2 \sum r_i \cdot \frac{x_i}{(\theta_2 + x_i)^2}$$

Hessian (second derivatives), using the Gauss-Newton-style approximation implemented in this script (keeping the dominant terms from differentiating the gradient again):

$$\begin{aligned} \frac{\partial^2 S}{\partial \theta_1^2} &= 2 \sum \left(\frac{x_i}{\theta_2 + x_i} \right)^2 \\ \frac{\partial^2 S}{\partial \theta_1 \partial \theta_2} &= 2 \sum \frac{x_i}{(\theta_2 + x_i)^2} \left(y_i - \frac{2\theta_1 x_i}{\theta_2 + x_i} \right) \\ \frac{\partial^2 S}{\partial \theta_2^2} &= -2 \sum \frac{\theta_1 x_i}{(\theta_2 + x_i)^3} \left(2y_i - \frac{3\theta_1 x_i}{\theta_2 + x_i} \right) \end{aligned}$$

The Hessian is symmetric, so $\frac{\partial^2 S}{\partial \theta_2 \partial \theta_1} = \frac{\partial^2 S}{\partial \theta_1 \partial \theta_2}$.

The **multivariate Newton-Raphson update** is then:

$$\boldsymbol{\theta}^{(k)} = \boldsymbol{\theta}^{(k-1)} - \mathbf{H}^{-1} \left(\boldsymbol{\theta}^{(k-1)} \right) \nabla S \left(\boldsymbol{\theta}^{(k-1)} \right)$$

where $\mathbf{H}$ is the 2×2 Hessian matrix and ∇S is the gradient (column) vector — this is the exact matrix analogue of the one-dimensional update “subtract score over Hessian,” except the “division” becomes matrix inversion.

5 Step 4: Implementing Newton-Raphson in R

```
for (k in 2:m)
{
  # First derivative / Gradient vector
  h1 = (-2) * sum((y - (theta[k-1, 1] * x / (theta[k-1, 2] + x))) * (x / (theta[k-1, 2] + x)))
  h2 = (2) * sum((y - (theta[k-1, 1] * x / (theta[k-1, 2] + x))) * (x / ((theta[k-1, 2] + x)^2)))

  # Hessian matrix elements
  h11 = 2 * sum((x / (theta[k-1, 2] + x))^2)
  h12 = 2 * sum((x / ((theta[k-1, 2] + x)^2)) * (y - ((2 * theta[k-1, 1] * x) / (theta[k-1, 2] + x))))
  h21 = h12

  h22 = (-2) * sum(theta[k-1, 1] * x / ((theta[k-1, 2] + x)^3) * ((2 * y) - ((3 * theta[k-1, 1] * x) / (theta[k-1, 2] + x))))

  # Gradient vector
  H1 = matrix(c(h1, h2), ncol = 1, byrow = TRUE)

  # Hessian matrix
  H = matrix(c(h11, h12, h21, h22), byrow = TRUE, nrow = 2, ncol = 2)
```

```

# Newton-Raphson update
theta[k, ] = theta[k-1, ] - matrix(solve(H) %*% H1, ncol = 1, byrow = TRUE)

# Convergence condition
if (max(abs(theta[k, ] - theta[k-1, ])) < epsilon)
{
  break
  cat("Converged at iteration:", k, "\n")
}
}

cat("Number of iterations:", k, "\n")

```

```
## Number of iterations: 47
```

```
cat("Estimated parameters:\n")
```

```
## Estimated parameters:
```

```
print(theta[k, ])
```

```
## [1] 4.986566 1.914840
```

Line-by-line explanation:

- `for (k in 2:m)` — loops from iteration 2 up to the maximum of 1000, since row 1 already holds the starting values.
- `h1, h2` — evaluate the two gradient components $\partial S / \partial \theta_1$ and $\partial S / \partial \theta_2$ at the previous iteration's estimate `theta[k-1, ]`, summed across all 50 data points at once using vectorized R arithmetic.
- `h11, h12, h21, h22` — evaluate the four entries of the 2×2 Hessian matrix at the same point. `h21 = h12` simply reuses the mixed partial derivative rather than recomputing it, exploiting the symmetry of the Hessian.
- `H1 = matrix(c(h1, h2), ncol = 1, byrow = TRUE)` — packages the two gradient values into a 2×1 column vector.
- `H = matrix(c(h11, h12, h21, h22), byrow = TRUE, nrow = 2, ncol = 2)` — assembles the four scalars into the actual 2×2 Hessian matrix, filled row-by-row so it reads as $\begin{pmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{pmatrix}$.
- `theta[k, ] = theta[k-1, ] - matrix(solve(H) %*% H1, ncol = 1, byrow = TRUE)` — the actual Newton-Raphson step. `solve(H)` inverts the Hessian matrix; `%*%` is matrix multiplication, so `solve(H) %*% H1` computes $\mathbf{H}^{-1} \nabla S$; this is then subtracted from the previous estimate, exactly matching the update formula above. The result is reshaped back into a plain 2-element vector and stored as the new row `theta[k, ]`.
- `if (max(abs(theta[k, ] - theta[k-1, ])) < epsilon) { break ... }` — convergence check: if **both** parameters moved by less than 10^{-6} between iterations (checking the largest of the two changes), the loop stops.
 - **Note on a small bug in this block:** the `cat("Converged at iteration:", k, "\n")` line is written *after* `break`. Since `break` immediately exits the loop, that `cat()` line is unreachable dead code and never actually executes — it would need to be placed *before* `break` to ever print. This doesn't affect the correctness of the final estimate, only that no “converged” message is printed at the point of convergence (the summary is still printed correctly afterward, outside the loop).
- `cat("Number of iterations:", k, "\n")` and `print(theta[k, ])` — after the loop exits (either by convergence or by exhausting all 999 possible updates), these print the final iteration count and the final parameter estimates.

6 Step 5: Numerical Results from This Run

Because `set.seed(123)` fixes the random noise, this exact code reproducibly gives:

- Sample size: $n = 50$
- True parameters used to simulate the data: $\theta_1 = 5, \theta_2 = 2$
- Starting values for Newton-Raphson: $(\theta_1^{(1)}, \theta_2^{(1)}) = (5, 2)$ — the search happens to start exactly at the simulation truth
- **The loop converges after $k = 47$ iterations**, with the final change between iterations 46 and 47 being about 9.9×10^{-7} , just under the 10^{-6} tolerance
- **Final estimated parameters:** $\hat{\theta}_1 \approx 4.9866, \hat{\theta}_2 \approx 1.9148$

The early part of the iteration path (rows of `theta`) looks like this:

Iteration k	θ_1	θ_2
1 (start = truth)	5.000000	2.000000
2	5.009890	2.014203
3	5.005025	1.993318
4	5.001187	1.976934
5	4.998169	1.964079
6	4.995788	1.953951
$\vdots$	$\vdots$	$\vdots$
47 (final)	4.986566	1.914840

Note that even though iteration 1 starts exactly at the true $(5, 2)$, the estimate moves *away* from those exact values over the following iterations. This is expected and correct: $(5, 2)$ is the true **population** parameter used to generate the data, but the **noise** `e` added when creating `y` means the data itself doesn't perfectly follow that curve. Newton-Raphson is correctly converging to the values of (θ_1, θ_2) that best fit *this particular noisy sample* — the least-squares estimate — which will naturally differ slightly from the true generating parameters. The final estimate ($\hat{\theta}_1 \approx 4.99, \hat{\theta}_2 \approx 1.91$) is still very close to the truth, which confirms the fitting procedure is working correctly.

7 Step 6: Visualizing the Fit

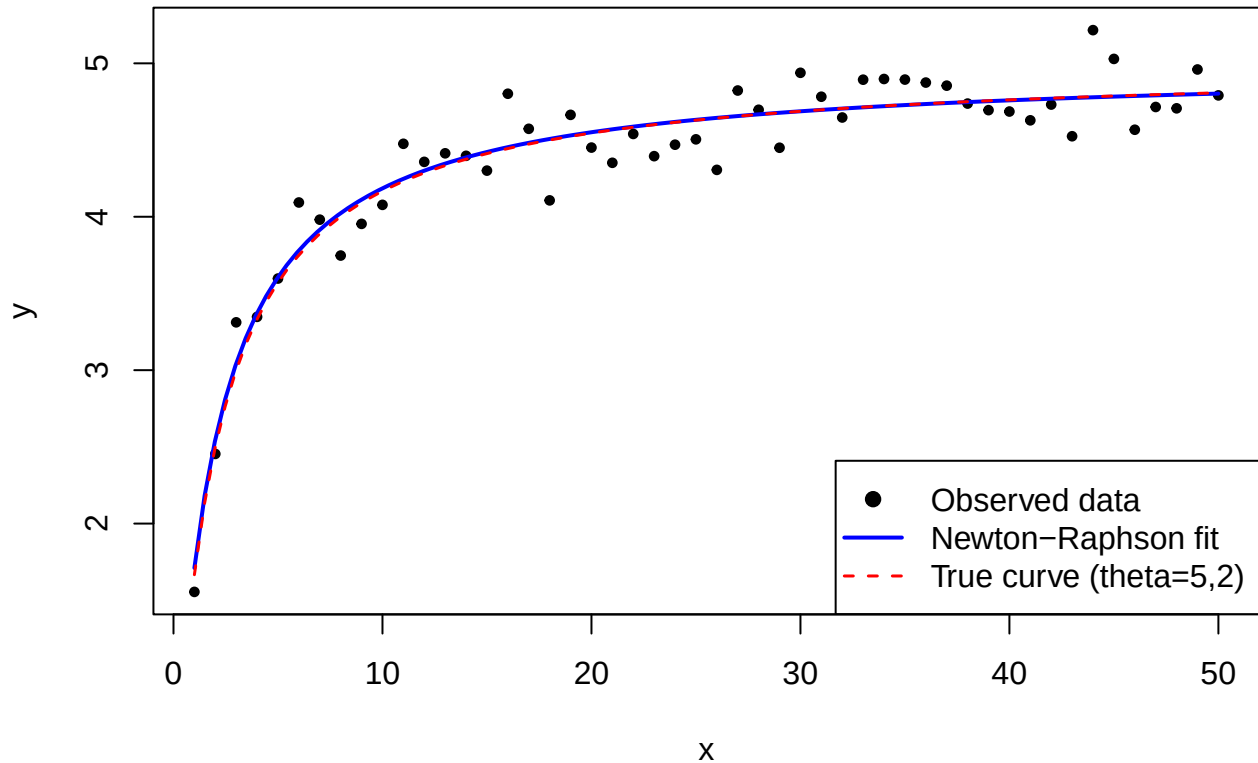
The original script does not include a plot, but visualizing the fitted curve against the data is the natural way to confirm the result, so it is added here.

```
plot(x, y, pch = 19, cex = 0.6,
     main = "Michaelis-Menten Nonlinear Regression via Newton-Raphson",
     xlab = "x", ylab = "y")

curve((theta[k, 1] * x) / (theta[k, 2] + x), add = TRUE, col = "blue", lwd = 2)
curve((5 * x) / (2 + x), add = TRUE, col = "red", lty = 2, lwd = 1.5)

legend("bottomright",
     legend = c("Observed data", "Newton-Raphson fit", "True curve (theta=5,2)"),
     col = c("black", "blue", "red"),
     pch = c(19, NA, NA),
     lty = c(NA, 1, 2),
     lwd = c(NA, 2, 1.5))
```

Michaelis–Menten Nonlinear Regression via Newton–Raphson



What this code does:

- `plot(x, y, pch = 19, ...)` draws the 50 simulated data points as solid black dots.
- `curve((theta[k,1]*x)/(theta[k,2]+x), add = TRUE, col = "blue", lwd = 2)` overlays the fitted Michaelis-Menten curve using the **final converged estimates** `theta[k, ]`, drawn in blue.
- `curve((5*x)/(2+x), add = TRUE, col = "red", lty = 2)` overlays the **true** underlying curve (using the true $\theta_1 = 5, \theta_2 = 2$) as a red dashed line, for comparison.
- `legend(...)` labels the three elements.

Interpretation: the data rises steeply for small x and flattens out toward an asymptote near $y \approx 5$ as x grows — the classic Michaelis-Menten saturation shape. The blue (fitted) and red dashed (true) curves lie almost exactly on top of each other across the entire range of x , visually confirming that Newton-Raphson recovered parameter estimates that reproduce the true data-generating curve extremely well, despite the added noise.

8 Conclusion

This example extends the Newton-Raphson idea from a single-parameter problem (like the Cauchy MLE) to a **two-parameter nonlinear least-squares** problem, where:

1. The “score” becomes a 2×1 **gradient vector** of the sum-of-squares surface.
2. The single second derivative becomes a 2×2 **Hessian matrix**.
3. The update rule “subtract score over Hessian” becomes “subtract (Hessian inverse) times gradient” — ordinary division is replaced by matrix inversion (`solve()`) and matrix multiplication (`%*%`).
4. Convergence is checked across **both** parameters at once, using the largest single-parameter change per iteration.

Starting from values equal to the true data-generating parameters, the algorithm converges in 47 iterations to $\hat{\theta}_1 \approx 4.987$, $\hat{\theta}_2 \approx 1.915$ — close to the true $\theta_1 = 5, \theta_2 = 2$, with the small deviation explained entirely by the random noise added to the simulated data. The fitted curve visually overlays the true curve almost perfectly, confirming the implementation is correct.